Gates & Circuit Design

Gates, Circuits and Reversible Logic

Quantum Computing — CEP

Kolin Paul

Department of Computer Science and Engineering
IIT Delhi

1 November 2026

From Yesterday to Today

What QC Foundations [3/4] gave us

- Kets, bras, outer products: $X = |0\rangle\langle 1| + |1\rangle\langle 0|$
- Gates are unitary: $U^\dagger U = I$
- One factor per wire, top wire first: $\mathrm{CNOT} = P_0 \otimes I + P_1 \otimes X$
- Phase kickback: $U_f |x\rangle |-\rangle = (-1)^{f(x)} |x\rangle |-\rangle$

Today

First: three models of computation

Part A — the toolbox: single-qubit and controlled gates, other measurement bases, classical control, identities

Part B — classical logic in a quantum circuit: reversible logic, oracles, ancillas, uncomputation

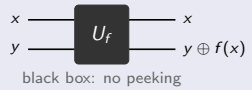
The question we end on

"Can a quantum computer add $3 + 4$?"

For every idea today, we will see it working in Qiskit.
Labs Page: https://www.cse.iitd.ac.in/~kolin/labs.html

Three Ways to Program a Quantum Computer

Query model (yesterday)

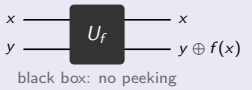


black box: no peeking

Count **oracle calls**.
Deutsch–Jozsa: 1 call vs $2^{n-1} + 1$.

Three Ways to Program a Quantum Computer

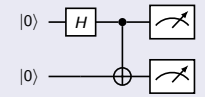
Query model (yesterday)



black box: no peeking

Count **oracle calls**.
Deutsch–Jozsa: 1 call vs $2^{n-1} + 1$.

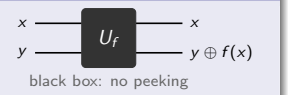
Circuit model (today)



Gates on wires, then measure.
Cost: gates, depth, qubits.
Universal: any computation.

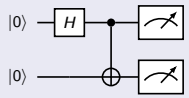
Three Ways to Program a Quantum Computer

Query model (yesterday)



Count **oracle calls**.
Deutsch–Jozsa: 1 call vs $2^{n-1}+1$.

Circuit model (today)



Gates on wires, then measure.
Cost: gates, depth, qubits.
Universal: any computation.

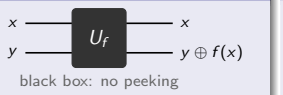
Annealing (D-Wave)



Write the problem as an **energy**; the hardware settles into a low-energy state.
A **heuristic optimiser**.

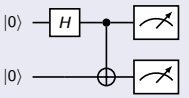
Three Ways to Program a Quantum Computer

Query model (yesterday)



Count **oracle calls**.
Deutsch–Jozsa: 1 call vs $2^{n-1}+1$.

Circuit model (today)



Gates on wires, then measure.
Cost: gates, depth, qubits.
Universal: any computation.

Annealing (D-Wave)



Write the problem as an **energy**; the hardware settles into a low-energy state.
A **heuristic optimiser**.

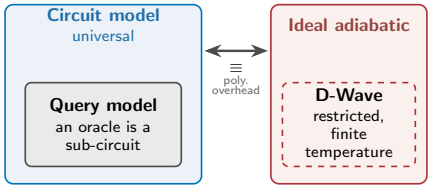
Same physics, different contracts: what you write, and what you pay for.

Kolin Paul (IIT Delhi) Gates & Circuit Design 1 November 2026 3 / 37

Side by Side

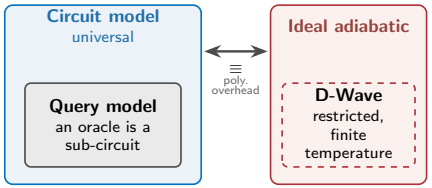
	Query (yesterday)	**Circuit** (today)	**Annealing** (D-Wave)
You write	an algorithm around a black box U_f	a sequence of gates on wires	an energy function (Ising / QUBO): $E(z) = \sum_i h_i z_i + \sum_{i<j} J_{ij} z_i z_j$
The machine	calls U_f	applies unitaries, then measures	morphs an easy Hamiltonian into yours, reads out a low-energy z
Cost	oracle calls	gates, depth, qubits	anneal time (set by the energy gap)
Gives you	provable speed-ups	universal computation	good answers, no guarantee
Hardware	an abstraction	IBM, Google, Quantinuum, IonQ	D-Wave Advantage, ∼5,000 qubits

How They Relate



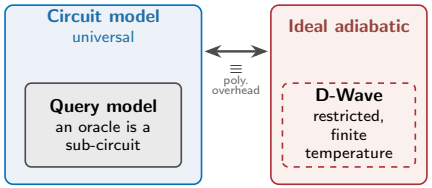
1. **Query lives inside circuit.** An oracle is just a sub-circuit; we build them in Part B today. Counting calls is how speed-ups get *proved*.

How They Relate



1. **Query lives inside circuit.** An oracle is just a sub-circuit; we build them in Part B today. Counting calls is how speed-ups get *proved*.
2. **Ideal adiabatic ≡ circuit.** Each simulates the other with polynomial overhead (Aharonov et al. 2007). D-Wave's machine is a restricted version: no proven speed-up, used as an optimiser.

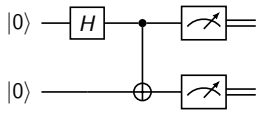
How They Relate



1. **Query lives inside circuit.** An oracle is just a sub-circuit; we build them in Part B today. Counting calls is how speed-ups get *proved*.
2. **Ideal adiabatic ≡ circuit.** Each simulates the other with polynomial overhead (Aharonov et al. 2007). D-Wave's machine is a restricted version: no proven speed-up, used as an optimiser.
3. **Qubit counts don't transfer.** ∼5,000 annealing qubits and 100+ gate-model qubits are different currencies. Optimisation on both kinds of machine comes later in the course.

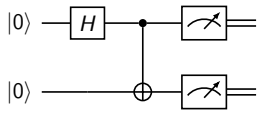
Other models: measurement-based (photonics), topological. All equivalent to circuits in principle.

The Circuit Model



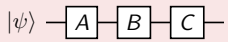
- **Wire** = one qubit, starts in $|0\rangle$
- **Time** runs left to right
- **Box** = a unitary gate
- **Meter** = measurement in the Z basis; the result is a classical bit (double wire)

The Circuit Model



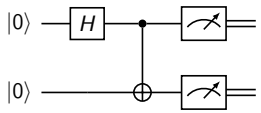
- **Wire** = one qubit, starts in $|0\rangle$
- **Time** runs left to right
- **Box** = a unitary gate
- **Meter** = measurement in the Z basis; the result is a classical bit (double wire)

Read left to right, multiply right to left



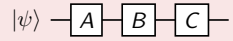
$$= C\,B\,A\,|\psi\rangle$$

The Circuit Model



- **Wire** = one qubit, starts in $|0\rangle$
- **Time** runs left to right
- **Box** = a unitary gate
- **Meter** = measurement in the Z basis; the result is a classical bit (double wire)

Read left to right, multiply right to left



$$= C\,B\,A\,|\psi\rangle$$

Order matters

S then H: $HS\,|0\rangle = H\,|0\rangle = |+\rangle$

H then S: $SH\,|0\rangle = S\,|+\rangle = |+i\rangle$

Notebook: §1

The Pauli Gates

Gate	Matrix	On $\|0\rangle, \|1\rangle$	Bloch sphere
X	$\left(\begin{smallmatrix}0&1\\1&0\end{smallmatrix}\right)$	$\|0\rangle \leftrightarrow \|1\rangle$	half-turn about x
Y	$\left(\begin{smallmatrix}0&-i\\i&0\end{smallmatrix}\right)$	$\|0\rangle \to i\,\|1\rangle,\ \|1\rangle \to -i\,\|0\rangle$	half-turn about y
Z	$\left(\begin{smallmatrix}1&0\\0&-1\end{smallmatrix}\right)$	$\|0\rangle \to \|0\rangle,\ \|1\rangle \to -\|1\rangle$	half-turn about z

The Pauli Gates

Gate	Matrix	On $\|0\rangle, \|1\rangle$	Bloch sphere
X	$\left(\begin{smallmatrix}0&1\\1&0\end{smallmatrix}\right)$	$\|0\rangle \leftrightarrow \|1\rangle$	half-turn about x
Y	$\left(\begin{smallmatrix}0&-i\\i&0\end{smallmatrix}\right)$	$\|0\rangle \to i\,\|1\rangle,\ \|1\rangle \to -i\,\|0\rangle$	half-turn about y
Z	$\left(\begin{smallmatrix}1&0\\0&-1\end{smallmatrix}\right)$	$\|0\rangle \to \|0\rangle,\ \|1\rangle \to -\|1\rangle$	half-turn about z

Algebra

$X^2 = Y^2 = Z^2 = I$

$XY = iZ, \quad YZ = iX, \quad ZX = iY$

Any two anticommute: $XZ = -ZX$

X is the classical NOT. Z has no classical meaning at all: it only changes a relative phase, which the Z basis cannot see (QC Foundations [3/4]).

The Hadamard Gate

$$H = \tfrac{1}{\sqrt{2}}\begin{pmatrix}1 & 1\\ 1 & -1\end{pmatrix} = \frac{X+Z}{\sqrt{2}}$$

$$H\,|0\rangle = |+\rangle, \quad H\,|1\rangle = |-\rangle, \quad H\,|\pm\rangle = |0\rangle, |1\rangle$$

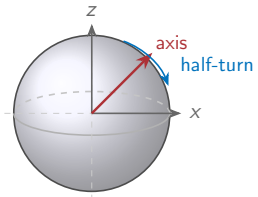
The Hadamard Gate

$$H = \frac{1}{\sqrt{2}}\begin{pmatrix}1 & 1\\ 1 & -1\end{pmatrix} = \frac{X+Z}{\sqrt{2}}$$

$$H|0\rangle = |+\rangle, \quad H|1\rangle = |-\rangle, \quad H|\pm\rangle = |0\rangle, |1\rangle$$

It swaps the Z and X bases

$H^2 = I$. That is why H appears just before every measurement in Deutsch–Jozsa: it turns an X-basis question into a Z-basis measurement.



A half-turn about the axis halfway between x and z: $z \leftrightarrow x$, and $y \to -y$.

The Phase Ladder: Z, S, T

One family: $P(\varphi) = \begin{pmatrix}1 & 0\\ 0 & e^{i\varphi}\end{pmatrix}$

$Z = P(\pi), \quad S = P(\pi/2), \quad T = P(\pi/4)$

$T^2 = S, \quad S^2 = Z, \quad Z^2 = I \quad (\text{so } T^8 = I)$

The Phase Ladder: Z, S, T

One family: $P(\varphi) = \begin{pmatrix}1 & 0\\ 0 & e^{i\varphi}\end{pmatrix}$

$Z = P(\pi), \quad S = P(\pi/2), \quad T = P(\pi/4)$

$T^2 = S, \quad S^2 = Z, \quad Z^2 = I \quad (\text{so } T^8 = I)$

What they do

Probabilities of 0/1 never change; the arrow turns about z by φ.

Also called the "$\pi/8$ gate": $T = e^{i\pi/8}\,\mathrm{diag}(e^{-i\pi/8}, e^{i\pi/8})$.

The Phase Ladder: Z, S, T

One family: $P(\varphi) = \begin{pmatrix}1 & 0\\ 0 & e^{i\varphi}\end{pmatrix}$

$Z = P(\pi), \quad S = P(\pi/2), \quad T = P(\pi/4)$

$T^2 = S, \quad S^2 = Z, \quad Z^2 = I \quad (\text{so } T^8 = I)$

What they do

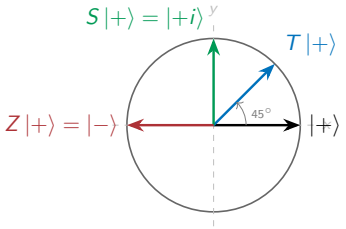
Probabilities of 0/1 never change; the arrow turns about z by φ.

Also called the "$\pi/8$ gate": $T = e^{i\pi/8}\,\mathrm{diag}(e^{-i\pi/8}, e^{i\pi/8})$.

```
qc = QuantumCircuit(1); qc.h(0); qc.t(0)
Statevector(qc)   # [0.707, 0.5+0.5j]
```

Notebook: §2

Looking down on the equator from $+z$



Rotation Gates

Turn by θ about an axis

$$R_x(\theta) = e^{-i\theta X/2} = \cos\tfrac{\theta}{2}\, I - i \sin\tfrac{\theta}{2}\, X$$

and the same for R_y, R_z.

$$R_y(\theta) = \begin{pmatrix} \cos\frac{\theta}{2} & -\sin\frac{\theta}{2} \\ \sin\frac{\theta}{2} & \cos\frac{\theta}{2} \end{pmatrix}, \qquad R_z(\theta) = \begin{pmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{pmatrix}$$

Why $\theta/2$? The Bloch angle is twice the angle in the ket.

Rotation Gates

Turn by θ about an axis

$$R_x(\theta) = e^{-i\theta X/2} = \cos\tfrac{\theta}{2}\, I - i \sin\tfrac{\theta}{2}\, X$$

and the same for R_y, R_z.

Two facts you will use

$R_y(\theta)\,|0\rangle = \cos\frac{\theta}{2}\,|0\rangle + \sin\frac{\theta}{2}\,|1\rangle$: any real-amplitude state.

$R_z(\theta) = e^{-i\theta/2} P(\theta)$: same gate as the phase gate, up to a global phase.

$$R_y(\theta) = \begin{pmatrix} \cos\frac{\theta}{2} & -\sin\frac{\theta}{2} \\ \sin\frac{\theta}{2} & \cos\frac{\theta}{2} \end{pmatrix}, \qquad R_z(\theta) = \begin{pmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{pmatrix}$$

Why $\theta/2$? The Bloch angle is twice the angle in the ket.

Rotation Gates

Turn by θ about an axis

$$R_x(\theta) = e^{-i\theta X/2} = \cos\tfrac{\theta}{2}\, I - i \sin\tfrac{\theta}{2}\, X$$

and the same for R_y, R_z.

Two facts you will use

$R_y(\theta)\,|0\rangle = \cos\frac{\theta}{2}\,|0\rangle + \sin\frac{\theta}{2}\,|1\rangle$: any real-amplitude state.

$R_z(\theta) = e^{-i\theta/2} P(\theta)$: same gate as the phase gate, up to a global phase.

$$R_y(\theta) = \begin{pmatrix} \cos\frac{\theta}{2} & -\sin\frac{\theta}{2} \\ \sin\frac{\theta}{2} & \cos\frac{\theta}{2} \end{pmatrix}, \qquad R_z(\theta) = \begin{pmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{pmatrix}$$

Why $\theta/2$? The Bloch angle is twice the angle in the ket.

Gate	Qiskit
X, Y, Z, H	`x y z h`
$S, S^\dagger, T, T^\dagger$	`s sdg t tdg`
R_x, R_y, R_z, P	`rx ry rz p`

Poll — Same Gate or Not?

Which pairs are the same gate, up to a global phase?

(A) S and $R_z(\pi/2)$ (B) Z and $R_z(\pi)$

(C) X and $R_x(\pi)$ (D) H and $R_y(\pi/2)$

Poll — Same Gate or Not?

Which pairs are the same gate, up to a global phase?

(A) S and $R_z(\pi/2)$ (B) Z and $R_z(\pi)$

(C) X and $R_x(\pi)$ (D) H and $R_y(\pi/2)$

Answer: A, B, C yes; D no

$R_z(\pi/2) = e^{-i\pi/4} S$, $R_z(\pi) = -iZ$, $R_x(\pi) = -iX$.

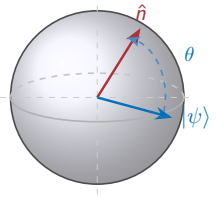
(D) agrees on $|0\rangle$ (both give $|+\rangle$), but $R_y(\pi/2)\,|1\rangle = -|-\rangle$ while $H\,|1\rangle = +|-\rangle$: a *relative* sign. In fact $H = X\,R_y(\pi/2)$.

```
Operator(h).equiv(Operator(ry))
# False
Operator(h).equiv(Operator(ry_then_x))
# True  (equiv ignores global phase)
```

Notebook: §3

Agreeing on one input proves nothing. Compare the whole matrix.

Every Single-Qubit Gate Is a Rotation



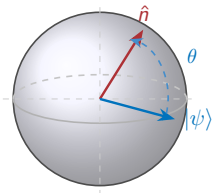
Fact

Every 2×2 unitary is

$$U = e^{i\alpha}\, R_{\hat{n}}(\theta)$$

a global phase times a turn by θ about some axis $\hat{n}$.

Every Single-Qubit Gate Is a Rotation



Fact

Every 2×2 unitary is

$$U = e^{i\alpha}\, R_{\hat{n}}(\theta)$$

a global phase times a turn by θ about some axis $\hat{n}$.

Gate	Axis	Angle
X, Y, Z	x, y, z	180°
H	$(x+z)/\sqrt{2}$	180°
S, T	z	90°, 45°

Next session: any such turn can be built from R_z, R_y, R_z (the ZYZ decomposition).

CNOT and CZ

CZ: flip the sign of $|11\rangle$

$$\mathrm{CZ} = \mathrm{diag}(1, 1, 1, -1) = P_0 \otimes I + P_1 \otimes Z$$

It is also $I \otimes P_0 + Z \otimes P_1$: **symmetric**, so it is drawn with two dots.



CNOT and CZ

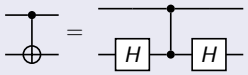
CZ: flip the sign of $|11\rangle$

$$\mathrm{CZ} = \mathrm{diag}(1,1,1,-1) = P_0 \otimes I + P_1 \otimes Z$$

It is also $I \otimes P_0 + Z \otimes P_1$: **symmetric**, so it is drawn with two dots.



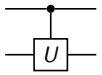
CNOT from CZ



Why: $HZH = X$ on the target.

Several chips have CZ, not CNOT, as their native two-qubit gate. The compiler inserts these H's for you (session [3/3]).

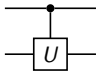
Controlled-U — and Where Kickback Comes From



$$CU = P_0 \otimes I + P_1 \otimes U = \begin{pmatrix} I & 0 \\ 0 & U \end{pmatrix}$$

Controlled-phase: $CP(\varphi) = \mathrm{diag}(1,1,1,e^{i\varphi})$, symmetric like CZ.

Controlled-U — and Where Kickback Comes From



$$CU = P_0 \otimes I + P_1 \otimes U = \begin{pmatrix} I & 0 \\ 0 & U \end{pmatrix}$$

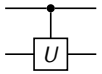
Controlled-phase: $CP(\varphi) = \mathrm{diag}(1,1,1,e^{i\varphi})$, symmetric like CZ.

If the target is an eigenvector, $U|u\rangle = e^{i\varphi}|u\rangle$

$$CU\big(\alpha|0\rangle + \beta|1\rangle\big)|u\rangle = \big(\alpha|0\rangle + \beta e^{i\varphi}|1\rangle\big)|u\rangle$$

The target is unchanged; the **phase moves to the control**.

Controlled-U — and Where Kickback Comes From



$$CU = P_0 \otimes I + P_1 \otimes U = \begin{pmatrix} I & 0 \\ 0 & U \end{pmatrix}$$

Controlled-phase: $CP(\varphi) = \mathrm{diag}(1,1,1,e^{i\varphi})$, symmetric like CZ.

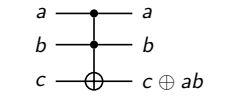
If the target is an eigenvector, $U|u\rangle = e^{i\varphi}|u\rangle$

$$CU\big(\alpha|0\rangle + \beta|1\rangle\big)|u\rangle = \big(\alpha|0\rangle + \beta e^{i\varphi}|1\rangle\big)|u\rangle$$

The target is unchanged; the **phase moves to the control**.

Deutsch–Jozsa used $U = X$, $|u\rangle = |-\rangle$, $e^{i\varphi} = -1$. Phase estimation (inside Shor) is this same move with a cleverer φ.

Toffoli and Multi-Controlled Gates



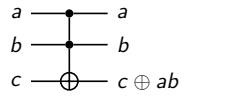
Toffoli = CCX = CCNOT

Flip the target only if both controls are 1

As an 8×8 matrix: the identity, except that it swaps $|110\rangle \leftrightarrow |111\rangle$.

Recipe: $(I - P_1 \otimes P_1) \otimes I + P_1 \otimes P_1 \otimes X$.

Toffoli and Multi-Controlled Gates



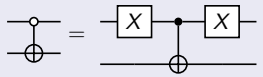
Toffoli = CCX = CCNOT

Flip the target only if both controls are 1

As an 8×8 matrix: the identity, except that it swaps $|110\rangle \leftrightarrow |111\rangle$.

Recipe: $(I - P_1 \otimes P_1) \otimes I + P_1 \otimes P_1 \otimes X$.

Variations



Open circle: control on 0. n controls: C^{n}X (`mcx` in Qiskit).

No hardware has a native Toffoli. What it costs is the first thing we do next session.

Measuring in Another Basis

Hardware only measures Z. So rotate first.

To measure in the basis $\{U|0\rangle, U|1\rangle\}$: apply $U^\dagger$, then measure.

Why: $P(k) = |\langle k| U^\dagger |\psi\rangle|^2 = |(U|k\rangle)^\dagger |\psi\rangle|^2$, the Born rule in the new basis.

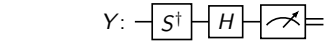
Measuring in Another Basis

Hardware only measures Z. So rotate first.

To measure in the basis $\{U|0\rangle, U|1\rangle\}$: apply $U^\dagger$, then measure.

Why: $P(k) = |\langle k| U^\dagger |\psi\rangle|^2 = |(U|k\rangle)^\dagger |\psi\rangle|^2$, the Born rule in the new basis.

Basis	States	Apply first
Z	$\|0\rangle, \|1\rangle$	nothing
X	$\|+\rangle, \|-\rangle$	H
Y	$\|+i\rangle, \|-i\rangle$	$S^\dagger$, then H



($|\pm i\rangle = SH|0/1\rangle$, so $U = SH$ and $U^\dagger = HS^\dagger$.)

Same State, Different Questions

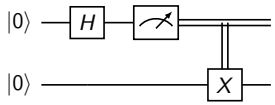
State	Z basis	X basis	Y basis
$\|+\rangle$	50/50	**always 0**	50/50
$\|+i\rangle$	50/50	50/50	**always 0**

In the Z basis the two states look identical. Ask the right question and each one is certain.

```
qc = QuantumCircuit(1, 1)
qc.h(0); qc.s(0)          # prepare |+i>
qc.sdg(0); qc.h(0)        # rotate: Y basis -> Z
qc.measure(0, 0)
AerSimulator().run(qc, shots=2000) \
    .result().get_counts()     # {'0': 2000}
```

Notebook: §4 (all six cases)

Classically Controlled Gates

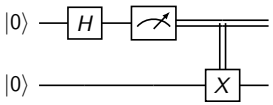


Measure, then apply X only if the bit was 1.

```
qc.measure(0, 0)
with qc.if_test((qc.clbits[0], 1)):
    qc.x(1)
```

Notebook: §5

Classically Controlled Gates



Measure, then apply X only if the bit was 1.

```
qc.measure(0, 0)
with qc.if_test((qc.clbits[0], 1)):
    qc.x(1)
```

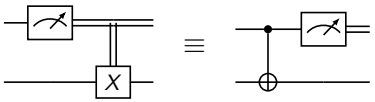
Notebook: §5

Where you will see this

- **Teleportation**: the receiver applies $X^{m_2}Z^{m_1}$ from two measured bits (QC Foundations [4/4])
- **Error correction**: measure a syndrome, then fix
- **Reset and reuse**: measure, flip back to $|0\rangle$

The gate is chosen while the circuit runs. On hardware this needs fast classical electronics next to the chip.

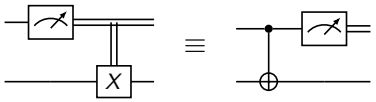
The Deferred Measurement Principle



Principle

A measurement followed by a classically controlled gate gives the same statistics as the quantum-controlled gate followed by the measurement.

The Deferred Measurement Principle

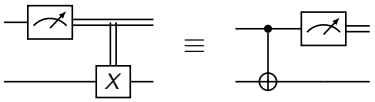


Principle

A measurement followed by a classically controlled gate gives the same statistics as the quantum-controlled gate followed by the measurement.

Notebook §5 runs both from $H|0\rangle$: each gives 00 or 11, half the time each.

The Deferred Measurement Principle



Principle

A measurement followed by a classically controlled gate gives the same statistics as the quantum-controlled gate followed by the measurement.

So

In theory, assume every measurement is at the end.

On hardware, measuring early lets you reset and reuse a qubit: fewer qubits.

Notebook §5 runs both from $H|0\rangle$: each gives 00 or 11, half the time each.

Circuit Identities I — One Wire

Self-inverse

$XX = YY = ZZ = HH = I$

Conjugation by H swaps X and Z

$HXH = Z, \quad HZH = X, \quad HYH = -Y$

The phase ladder

$TT = S, \quad SS = Z, \quad SS^\dagger = I, \quad SXS^\dagger = Y$

Circuit Identities I — One Wire

Self-inverse

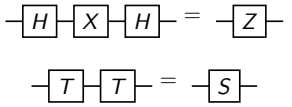
$XX = YY = ZZ = HH = I$

Conjugation by H swaps X and Z

$HXH = Z, \quad HZH = X, \quad HYH = -Y$

The phase ladder

$TT = S, \quad SS = Z, \quad SS^\dagger = I, \quad SXS^\dagger = Y$

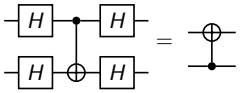


Remember the order: the matrix $SXS^\dagger$ is the circuit $S^\dagger$, then X, then S.

A compiler's optimisation pass looks for exactly these patterns (session [3/3]).

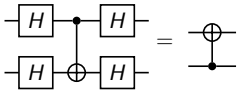
Circuit Identities II — Two Wires

Turn a CNOT round

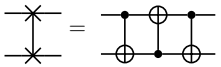


Circuit Identities II — Two Wires

Turn a CNOT round

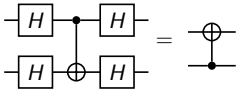


SWAP from three CNOTs

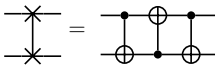


Circuit Identities II — Two Wires

Turn a CNOT round



SWAP from three CNOTs

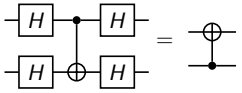


$$|a, b\rangle \xrightarrow{\mathrm{CX}_{12}} |a,\ a \oplus b\rangle \xrightarrow{\mathrm{CX}_{21}} |a \oplus a \oplus b,\ a \oplus b\rangle = |b,\ a \oplus b\rangle \xrightarrow{\mathrm{CX}_{12}} |b,\ b \oplus a \oplus b\rangle = |b,\ a\rangle$$

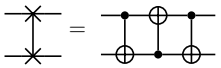
True on every basis state, so by linearity on every state.

Circuit Identities II — Two Wires

Turn a CNOT round



SWAP from three CNOTs

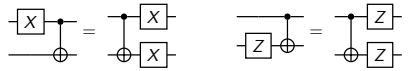


$$|a, b\rangle \xrightarrow{\mathrm{CX}_{12}} |a,\ a \oplus b\rangle \xrightarrow{\mathrm{CX}_{21}} |a \oplus a \oplus b,\ a \oplus b\rangle = |b,\ a \oplus b\rangle \xrightarrow{\mathrm{CX}_{12}} |b,\ b \oplus a \oplus b\rangle = |b,\ a\rangle$$

True on every basis state, so by linearity on every state.

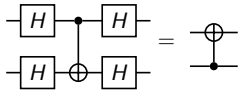
Pushing Paulis through a CNOT

Spread:

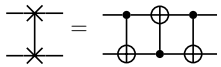


Circuit Identities II — Two Wires

Turn a CNOT round



SWAP from three CNOTs

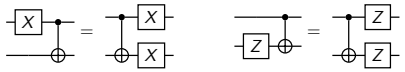


$$|a, b\rangle \xrightarrow{\mathrm{CX}_{12}} |a,\ a \oplus b\rangle$$
$$\xrightarrow{\mathrm{CX}_{21}} |a \oplus a \oplus b,\ a \oplus b\rangle = |b,\ a \oplus b\rangle$$
$$\xrightarrow{\mathrm{CX}_{12}} |b,\ b \oplus a \oplus b\rangle = |b,\ a\rangle$$

True on every basis state, so by linearity on every state.

Pushing Paulis through a CNOT

Spread:



Pass straight through:



X spreads from control to target; Z from target to control.

Check It Yourself — `Operator.equiv`

```
from qiskit import QuantumCircuit
from qiskit.quantum_info import Operator

a = QuantumCircuit(2)
a.h([0, 1]); a.cx(0, 1); a.h([0, 1])
b = QuantumCircuit(2)
b.cx(1, 0)
Operator(a).equiv(Operator(b))   # True
```

Notebook: §6

`equiv` ignores global phase, so HYH and Y come out "equivalent". Use == on the operators for an exact match.

Try (5 minutes)

1. Three CNOTs make a SWAP.
2. X before a CNOT's control equals $X \otimes X$ after it.
3. CNOT $= (I \otimes H)\,\mathrm{CZ}\,(I \otimes H)$.
4. H vs $R_y(\pi/2)$, then $R_y(\pi/2)$ followed by X.

Qiskit numbers qubits from the bottom of its printed bit strings ($q_1 q_0$); `equiv` does not care.

Break

10 minutes

After the break: making classical logic reversible, and what that has to do with oracles.

Why AND Cannot Be a Quantum Gate

Quantum gates are unitary, so they are **invertible**: different inputs give different outputs.

AND is **many-to-one**:

$$00,\ 01,\ 10 \longmapsto 0$$

Why AND Cannot Be a Quantum Gate

Quantum gates are unitary, so they are **invertible**: different inputs give different outputs.

AND is **many-to-one**:

$$00,\ 01,\ 10 \longmapsto 0$$

Seeing the output 0, you cannot tell which input it came from. Information is erased, so there is no inverse and no unitary.

Why AND Cannot Be a Quantum Gate

Quantum gates are unitary, so they are **invertible**: different inputs give different outputs.

AND is **many-to-one**:

$$00,\ 01,\ 10 \longmapsto 0$$

Seeing the output 0, you cannot tell which input it came from. Information is erased, so there is no inverse and no unitary.

The fix (Bennett, 1973)

Keep the inputs. Write the answer into a fresh bit:

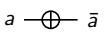
$$(a, b, c) \mapsto (a,\ b,\ c \oplus ab)$$

This is a permutation of the 8 inputs, so it *is* reversible. It is the Toffoli gate.

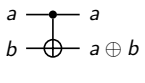
Landauer: erasing a bit costs at least $k_B T \ln 2$ of energy. Reversible logic erases nothing.

The Reversible Gate Zoo

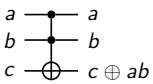
NOT



CNOT



Toffoli



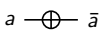
Fredkin



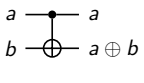
swap x, y if $c = 1$

The Reversible Gate Zoo

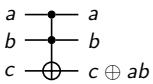
NOT



CNOT



Toffoli



Fredkin



swap x, y if $c = 1$

Universality (classical)

With a few extra bits fixed to 0 or 1, **Toffoli alone** can compute any Boolean function reversibly. So can Fredkin.

Every gate here is its own inverse: run it twice and you are back where you started.

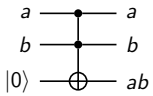
Boolean Gates from Toffoli + Ancillas

An **ancilla** is an extra qubit prepared in a known state.

Want	Build	Result
NOT	X	$\bar{a}$
XOR	CNOT	$a \oplus b$
AND	Toffoli onto $\lvert 0\rangle$	ab
NAND	Toffoli onto $\lvert 1\rangle$	$\overline{ab}$
OR	X on a, b; Toffoli onto $\lvert 1\rangle$; X on a, b	$a \vee b$
copy	CNOT onto $\lvert 0\rangle$	a, a

OR: $1 \oplus \bar{a}\bar{b} = a \vee b$ (De Morgan).

AND into a fresh ancilla



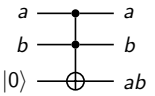
Boolean Gates from Toffoli + Ancillas

An **ancilla** is an extra qubit prepared in a known state.

Want	Build	Result
NOT	X	$\bar{a}$
XOR	CNOT	$a \oplus b$
AND	Toffoli onto $\lvert 0\rangle$	ab
NAND	Toffoli onto $\lvert 1\rangle$	$\overline{ab}$
OR	X on a, b; Toffoli onto $\lvert 1\rangle$; X on a, b	$a \vee b$
copy	CNOT onto $\lvert 0\rangle$	a, a

OR: $1 \oplus \bar{a}\bar{b} = a \vee b$ (De Morgan).

AND into a fresh ancilla



Copying ≠ cloning

CNOT copies a *basis* bit: $\lvert a\rangle \lvert 0\rangle \to \lvert a\rangle \lvert a\rangle$. On a superposition it makes an entangled state, not two copies.

Kolin Paul (IIT Delhi) Gates & Circuit Design 1 November 2026 26 / 37

Reversible Synthesis: One Toffoli per Term

Recipe for $\lvert x\rangle \lvert y\rangle \mapsto \lvert x\rangle \lvert y \oplus f(x)\rangle$

1. Write f as an XOR of AND terms (an ESOP).
2. One multi-controlled X per term, all onto y.

Reversible Synthesis: One Toffoli per Term

Recipe for $\lvert x\rangle \lvert y\rangle \mapsto \lvert x\rangle \lvert y \oplus f(x)\rangle$

1. Write f as an XOR of AND terms (an ESOP).
2. One multi-controlled X per term, all onto y.

Example: majority of three bits

$$\mathrm{MAJ}(x_1, x_2, x_3) = x_1x_2 \oplus x_1x_3 \oplus x_2x_3$$

Check 111: $1 \oplus 1 \oplus 1 = 1$. 110: $1 \oplus 0 \oplus 0 = 1$. 100: 0.

Kolin Paul (IIT Delhi) Gates & Circuit Design 1 November 2026 27 / 37

Reversible Synthesis: One Toffoli per Term

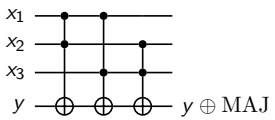
Recipe for $|x\rangle\,|y\rangle \mapsto |x\rangle\,|y \oplus f(x)\rangle$

1. Write f as an XOR of AND terms (an ESOP).
2. One multi-controlled X per term, all onto y.

Example: majority of three bits

$$\mathrm{MAJ}(x_1, x_2, x_3) = x_1x_2 \oplus x_1x_3 \oplus x_2x_3$$

Check 111: $1 \oplus 1 \oplus 1 = 1$. 110: $1 \oplus 0 \oplus 0 = 1$. 100: 0.



No ancilla needed. Notebook §7 checks all 8 inputs.

Real synthesis tools search for cheaper circuits, but the output is always a list of multi-controlled X's like this one.

The Garbage Problem

Longer functions need scratch bits. A careless circuit leaves them dirty:

$$|x\rangle\,|0\rangle\,|0\rangle \;\longmapsto\; |x\rangle\,|g(x)\rangle\,|f(x)\rangle$$

The Garbage Problem

Longer functions need scratch bits. A careless circuit leaves them dirty:

$$|x\rangle\,|0\rangle\,|0\rangle \;\longmapsto\; |x\rangle\,|g(x)\rangle\,|f(x)\rangle$$

Run Deutsch–Jozsa with this oracle. After the kickback:

$$\sum_x (-1)^{f(x)}\,|x\rangle\;|g(x)\rangle$$

The Garbage Problem

Longer functions need scratch bits. A careless circuit leaves them dirty:

$$|x\rangle\,|0\rangle\,|0\rangle \;\longmapsto\; |x\rangle\,|g(x)\rangle\,|f(x)\rangle$$

Run Deutsch–Jozsa with this oracle. After the kickback:

$$\sum_x (-1)^{f(x)}\,|x\rangle\;|g(x)\rangle$$

If $g(x)$ differs for different x, the branches are **labelled**. The final H's can no longer make them interfere.

The Garbage Problem

Longer functions need scratch bits. A careless circuit leaves them dirty:

$$|x\rangle\,|0\rangle\,|0\rangle \;\longmapsto\; |x\rangle\,|g(x)\rangle\,|f(x)\rangle$$

Run Deutsch–Jozsa with this oracle. After the kickback:

$$\sum_x (-1)^{f(x)}\,|x\rangle\;|g(x)\rangle$$

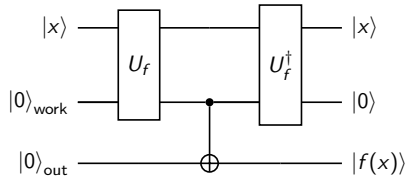
If $g(x)$ differs for different x, the branches are **labelled**. The final H's can no longer make them interfere.

Worst case: $g(x) = x$

The state is $\sum_x (-1)^{f(x)}\,|x\rangle\,|x\rangle$. The top register alone is completely random, so the algorithm reads noise for *every* f.

Garbage is "which-path" information. Interference needs the paths to be indistinguishable.

Compute — Copy — Uncompute



1. **Compute** f into a work register (with garbage).
2. **Copy** the answer out with a CNOT.
3. **Uncompute**: run the same gates in reverse order.

Each Toffoli and CNOT is its own inverse, so $U_f^\dagger$ is just U_f played backwards.

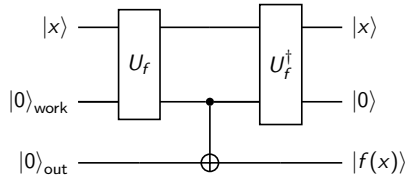
Result

$$|x\rangle\,|0\rangle\,|0\rangle \;\longmapsto\; |x\rangle\,|0\rangle\,|f(x)\rangle$$

Input kept, work register clean, answer out. Interference works again.

Price: about twice the gates. Clean answers are not free.

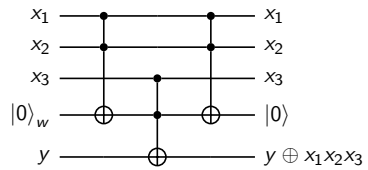
Compute — Copy — Uncompute



1. **Compute** f into a work register (with garbage).
2. **Copy** the answer out with a CNOT.
3. **Uncompute**: run the same gates in reverse order.

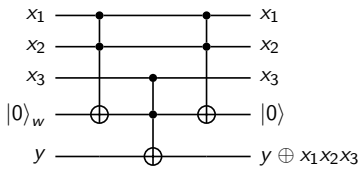
Each Toffoli and CNOT is its own inverse, so $U_f^\dagger$ is just U_f played backwards.

Worked: A 3-Input AND with One Ancilla



1. **Compute** $w = x_1x_2$
2. **Copy**: $y \oplus= w\,x_3$
3. **Uncompute**: $w \oplus= x_1x_2$, back to 0

Worked: A 3-Input AND with One Ancilla



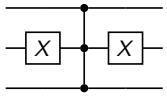
1. **Compute** $w = x_1x_2$
2. **Copy**: $y \oplus= w\,x_3$
3. **Uncompute**: $w \oplus= x_1x_2$, back to 0

3 Toffolis + 1 clean ancilla make a C^3X.

Notebook §10 runs all 8 inputs: the ancilla always comes back as 0.

Marking One State: $|101\rangle$

$f(x) = 1$ only for $x = 101$. Phase oracle:



The X's make the middle control fire on 0. The CCZ adds -1 only when all three wires are 1.

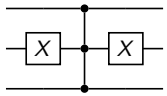
```
qc = QuantumCircuit(3); qc.h([0, 1, 2])
qc.x(1); qc.ccz(0, 1, 2); qc.x(1)
Statevector(qc)
# all +1/sqrt(8), except |101>: -1/sqrt(8)
```

Notebook: §9

This is the oracle Grover's search calls (Core Algorithms). "Marking" a state means flipping its sign.

Marking One State: $|101\rangle$

$f(x) = 1$ only for $x = 101$. Phase oracle:



The X's make the middle control fire on 0. The CCZ adds -1 only when all three wires are 1.

Ancillas: Clean, Dirty, and the Trade-off

Two kinds

Clean: starts in $|0\rangle$, must be returned as $|0\rangle$.

Dirty (borrowed): starts in an unknown state, must be returned *unchanged*. Any idle qubit can lend itself.

Ancillas: Clean, Dirty, and the Trade-off

Two kinds

Clean: starts in $|0\rangle$, must be returned as $|0\rangle$.

Dirty (borrowed): starts in an unknown state, must be returned *unchanged*. Any idle qubit can lend itself.

Qubits vs gates

A C^{n}X with $n-2$ clean ancillas (the "V-chain") takes $2n-3$ Toffolis. With no ancillas it is still possible, but needs many more gates.

Ancillas: Clean, Dirty, and the Trade-off

Two kinds

Clean: starts in $|0\rangle$, must be returned as $|0\rangle$.

Dirty (borrowed): starts in an unknown state, must be returned *unchanged*. Any idle qubit can lend itself.

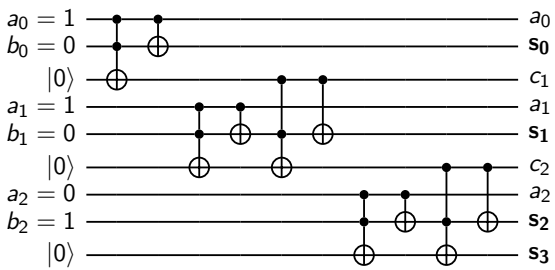
Qubits vs gates

A C^{n}X with $n-2$ clean ancillas (the "V-chain") takes $2n-3$ Toffolis. With no ancillas it is still possible, but needs many more gates.

Controls n	Clean ancillas	Toffolis
3	1	3
4	2	5
10	8	17

Qubits are scarce, gates are noisy. Every compiler weighs one against the other.

"Can I Compute $3+4$?" — Yes



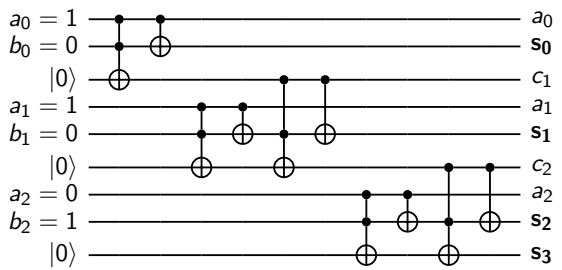
$a = 011 = 3$, $b = 100 = 4$

Bit 0: half adder (Toffoli, CNOT)

Bits 1, 2: full adders (Toffoli, CNOT, Toffoli, CNOT)

The sum overwrites b; the final carry is s_3.

"Can I Compute $3+4$?" — Yes



$a = 011 = 3$, $b = 100 = 4$

Bit 0: half adder (Toffoli, CNOT)

Bits 1, 2: full adders (Toffoli, CNOT, Toffoli, CNOT)

The sum overwrites b; the final carry is s_3.

Result

$s_3 s_2 s_1 s_0 = 0111 = 7$
in 100 out of 100 shots

Notebook: §11 (all 64 sums correct)

What Did That Cost?

The adder, counted

9 qubits (6 inputs, 3 carries)
5 Toffolis, 5 CNOTs
c_1, c_2 left as **garbage** (cleaning them costs more Toffolis)

What Did That Cost?

The adder, counted

9 qubits (6 inputs, 3 carries)
5 Toffolis, 5 CNOTs
c_1, c_2 left as **garbage** (cleaning them costs more Toffolis)

But no chip has a Toffoli

```
transpile(ccx_circuit,
  basis_gates=['cx','h','t','tdg'])
# {'cx': 6, 't': 4, 'tdg': 3, 'h': 2}
```

Notebook: §12

What Did That Cost?

The adder, counted

9 qubits (6 inputs, 3 carries)
5 Toffolis, 5 CNOTs
c_1, c_2 left as **garbage** (cleaning them costs more Toffolis)

So the "5 Toffolis" are really

$\approx$ 35 CNOTs and 35 T-type gates, before the chip's wiring adds SWAPs.

But no chip has a Toffoli

```
transpile(ccx_circuit,
  basis_gates=['cx','h','t','tdg'])
# {'cx': 6, 't': 4, 'tdg': 3, 'h': 2}
```

Notebook: §12

What Did That Cost?

The adder, counted

9 qubits (6 inputs, 3 carries)
5 Toffolis, 5 CNOTs
c_1, c_2 left as **garbage** (cleaning them costs more Toffolis)

So the "5 Toffolis" are really

$\approx$ 35 CNOTs and 35 T-type gates, before the chip's wiring adds SWAPs.

But no chip has a Toffoli

```
transpile(ccx_circuit,
  basis_gates=['cx','h','t','tdg'])
# {'cx': 6, 't': 4, 'tdg': 3, 'h': 2}
```

Notebook: §12

Universal $\neq$ useful

A classical adder does $3 + 4$ in a nanosecond for almost no energy. A quantum computer only wins on special problems (factoring, search, simulating molecules).

Next session: why T is the expensive gate, and how to count it.

Gate Card

Gate	Matrix	Bloch	Qiskit
X	$\left(\begin{smallmatrix}0&1\\1&0\end{smallmatrix}\right)$	π about x	`x`
Y	$\left(\begin{smallmatrix}0&-i\\i&0\end{smallmatrix}\right)$	π about y	`y`
Z	$\left(\begin{smallmatrix}1&0\\0&-1\end{smallmatrix}\right)$	π about z	`z`
H	$\frac{1}{\sqrt{2}}\left(\begin{smallmatrix}1&1\\1&-1\end{smallmatrix}\right)$	π about $\frac{x+z}{\sqrt{2}}$	`h`
S	$\left(\begin{smallmatrix}1&0\\0&i\end{smallmatrix}\right)$	$\pi/2$ about z	`s`
T	$\left(\begin{smallmatrix}1&0\\0&e^{i\pi/4}\end{smallmatrix}\right)$	$\pi/4$ about z	`t`
$R_n(\theta)$	$e^{-i\theta\,\hat{n}\cdot\vec{\sigma}/2}$	θ about $\hat{n}$	`rx ry rz`
$P(\varphi)$	$\left(\begin{smallmatrix}1&0\\0&e^{i\varphi}\end{smallmatrix}\right)$	φ about z	`p`

Gate	Action	Qiskit
CNOT	$\lvert a,b\rangle \to \lvert a, a\oplus b\rangle$	`cx`
CZ	$\lvert 11\rangle \to -\lvert 11\rangle$	`cz`
CU	$P_0\otimes I + P_1\otimes U$	`.control()`
SWAP	$\lvert a,b\rangle \to \lvert b,a\rangle$	`swap`
Toffoli	$\lvert a,b,c\rangle \to \lvert a,b,c\oplus ab\rangle$	`ccx`
C^{n}X	flip if all n are 1	`mcx`

Measure in basis $\{U\lvert k\rangle\}$: apply $U^\dagger$, then measure.
X: H. Y: $S^\dagger$ then H.

Uncompute: play the gates backwards.

Problem Set

1. Show $T^2 = S$ and $S^2 = Z$. What is the smallest $k \geq 1$ with $T^k = I$?
2. Apply the circuit H, then S, then H to $\lvert 0\rangle$ (mind the order). Which named state do you get, up to global phase?
3. Show that CZ is symmetric, and that $\mathrm{CNOT} = (I\otimes H)\,\mathrm{CZ}\,(I\otimes H)$.
4. Design a circuit that measures in the basis $\{\cos\frac{\pi}{8}\lvert 0\rangle + \sin\frac{\pi}{8}\lvert 1\rangle\,,\ -\sin\frac{\pi}{8}\lvert 0\rangle + \cos\frac{\pi}{8}\lvert 1\rangle\}$.
5. Prove the three identities of *Circuit Identities II* in Dirac notation: (a) turning a CNOT round; (b) SWAP from three CNOTs; (c) the four Pauli rules, e.g. $\mathrm{CNOT}\,(X\otimes I)\,\mathrm{CNOT} = X\otimes X$.
6. Write a bit oracle and a phase oracle for $f(x_1,x_2,x_3) = x_1x_3 \oplus x_2$.
7. Compute $a\vee b$ into a clean ancilla with one Toffoli and some X gates, leaving a and b unchanged.
8. Trace the adder for $3+5$. Which carry qubits end up as 1?

Check 2–7 with `Operator.equiv` or the simulator; the notebook has the building blocks.

Reading

Nielsen & Chuang

- §4.2: single-qubit operations
- §4.3: controlled operations
- §4.4: measurement (deferred measurement)
- §3.2.5: energy and reversible computation

Qiskit documentation

- `QuantumCircuit`: the standard gates
- `quantum_info.Operator`
- Classical feedforward: `if_test`

Run the notebook before next session. Change a gate, predict the output, then check.